

```

import matplotlib.pyplot as plt
import numpy as np

# =====
# Funções auxiliares
# =====

def gauss_jordan(A, b):
    """Resolve sistema linear Ax = b pelo método de Gauss-Jordan (manual)."""
    n = len(b)
    M = [A[i] + [b[i]] for i in range(n)]

    for i in range(n):
        # Pivoteamento
        if M[i][i] == 0:
            for k in range(i+1, n):
                if M[k][i] != 0:
                    M[i], M[k] = M[k], M[i]
                    break
        # Normaliza pivô
        pivot = M[i][i]
        M[i] = [m/pivot for m in M[i]]
        # Elimina coluna
        for j in range(n):
            if j != i:
                fator = M[j][i]
                M[j] = [M[j][k] - fator*M[i][k] for k in range(n+1)]

    return [M[i][-1] for i in range(n)]

def montar_matriz_equilibrio(N, NB, x, y, barra, Fx, Fy, apoios):
    """Monta matriz de equilíbrio para treliça 2D."""
    A = [[0]*(NB+sum(apoios)) for _ in range(2*N)]
    b = [0]*(2*N)

    # Forças aplicadas
    for i in range(N):
        b[2*i] = Fx[i]
        b[2*i+1] = Fy[i]

    # Barras
    for k in range(NB):
        i, j = barra[k]
        dx = x[j]-x[i]
        dy = y[j]-y[i]
        L = (dx**2+dy**2)**0.5
        cx, cy = dx/L, dy/L
        A[2*i][k] = cx

```

```

A[2*i+1][k] = cy
A[2*i][k] = -cx
A[2*j+1][k] = -cy

```

```

# Apoios
col = NB
for i in range(N):
    if apoios[i] == 1: # móvel (restrição em y)
        A[2*i+1][col] = 1
        col += 1
    elif apoios[i] == 2: # fixo (restrição em x e y)
        A[2*i][col] = 1
        col += 1
        A[2*i+1][col] = 1
        col += 1

return A, b

```

```

def classificar_barras(forcas):
    """Classifica barras como tração (+), compressão (-) ou nula."""
    return ["+" if f>1e-6 else "-" if f<-1e-6 else "0" for f in forcas]

```

```

# =====
# Entrada do usuário
# =====

```

```

N = int(input("Número de nós: "))
x, y = [], []
for i in range(N):
    xi = float(input(f"Coordenada x do nó {i}: "))
    yi = float(input(f"Coordenada y do nó {i}: "))
    x.append(xi)
    y.append(yi)

```

```

NB = int(input("Número de barras: "))
barra = []
for k in range(NB):
    i = int(input(f"Nó inicial da barra {k}: "))
    j = int(input(f"Nó final da barra {k}: "))
    barra.append((i,j))

```

```

Fx, Fy = [], []
for i in range(N):
    Fx.append(float(input(f"Força Fx no nó {i}: ")))
    Fy.append(float(input(f"Força Fy no nó {i}: ")))

```

```

apoios = []
print("Tipos de apoio: 0=livre, 1=móvel (restrição em y), 2=fixo (restrição em x e y)")

```

```

for i in range(N):
    apoios.append(int(input(f"Apoio no nó {i}: ")))

# =====
# Cálculos
# =====

A, b = montar_matriz_equilibrio(N, NB, x, y, barra, Fx, Fy, apoios)
sol = gauss_jordan([row[:] for row in A], b[:])

forcas_barras = sol[:NB]
classificacao = classificar_barras(forcas_barras)

print("\n--- Resultados ---")
print("Forças nas barras:", forcas_barras)
print("Classificação:", classificacao)
barra_max = np.argmax(np.abs(forcas_barras))
print("Barra mais solicitada:", barra_max)
print("Força média:", np.mean(forcas_barras))
print("Somatório das reações:", sum(sol[NB:]))

# =====
# Gráficos
# =====

# Treliza original
plt.figure(figsize=(10,4))
for k,(i,j) in enumerate(barra):
    plt.plot([x[i],x[j]], [y[i],y[j]], 'k-')
for i in range(N):
    plt.text(x[i],y[i]+0.05,f"N{i}")
plt.title("Treliza Original")
plt.axis('equal')
plt.show()

# Treliza colorida por intensidade
plt.figure(figsize=(10,4))
for k,(i,j) in enumerate(barra):
    cor = 'blue' if forcas_barras[k]>0 else 'red'
    plt.plot([x[i],x[j]], [y[i],y[j]], color=cor, linewidth=2+abs(forcas_barras[k]))
plt.title("Treliza Colorida por Força")
plt.axis('equal')
plt.show()

# Gráfico de barras
plt.figure()
plt.bar(range(NB), np.abs(forcas_barras))
plt.title("Magnitude das Forças em Cada Barra")

```

```
plt.xlabel("Barra")
plt.ylabel("Força")
plt.show()
```

```
# Histograma
plt.figure()
plt.hist(forcas_barras, bins=10)
plt.title("Distribuição das Forças Internas")
plt.xlabel("Força")
plt.ylabel("Frequência")
plt.show()
```

Arrumado:

```
import matplotlib.pyplot as plt
import numpy as np

# =====
# Funções auxiliares
# =====R

def gauss_jordan(A, b):
    """Resolve sistema linear  $Ax = b$  pelo método de Gauss-Jordan
    (manual)."""
    n = len(b)
    M = [A[i] + [b[i]] for i in range(n)]

    for i in range(n):
        # Pivoteamento
        if M[i][i] == 0:
            for k in range(i+1, n):
                if M[k][i] != 0:
                    M[i], M[k] = M[k], M[i]
                    break
        # Normaliza pivô
        pivot = M[i][i]
        M[i] = [m/pivot for m in M[i]]
        # Elimina coluna
        for j in range(n):
            if j != i:
                fator = M[j][i]
                M[j] = [M[j][k] - fator*M[i][k] for k in range(n+1)]

    return [M[i][-1] for i in range(n)]
```

```

def montar_matriz_equilibrio(N, NB, x, y, barra, Fx, Fy, apoios):
    """Monta matriz de equilíbrio para treliça 2D."""
    A = [[0]*(NB+sum(apoios)) for _ in range(2*N)]
    b = [0]*(2*N)

    # Forças aplicadas
    for i in range(N):
        b[2*i] = Fx[i]
        b[2*i+1] = Fy[i]

    # Barras
    for k in range(NB):
        i, j = barra[k]
        dx = x[j]-x[i]
        dy = y[j]-y[i]
        L = (dx**2+dy**2)**0.5
        cx, cy = dx/L, dy/L
        A[2*i][k] = cx
        A[2*i+1][k] = cy
        A[2*j][k] = -cx
        A[2*j+1][k] = -cy

    # Apoios
    col = NB
    for i in range(N):
        if apoios[i] == 1: # móvel (restrição em y)
            A[2*i+1][col] = 1
            col += 1
        elif apoios[i] == 2: # fixo (restrição em x e y)
            A[2*i][col] = 1
            col += 1
            A[2*i+1][col] = 1
            col += 1

    return A, b

def classificar_barras(forcas):
    """Classifica barras como tração (+), compressão (-) ou nula."""
    return ["+" if f>1e-6 else "-" if f<-1e-6 else "0" for f in forcas]

# =====
# Entrada do usuário
# =====

```

```

N = int(input("Número de nós: "))
x, y = [], []
for i in range(N):
    coords = input(f"Coordenadas do Nó {i} (x y): ").split()
    x.append(float(coords[0]))
    y.append(float(coords[1]))

NB = int(input("Número de barras: "))
barra = []
for k in range(NB):
    conn_str = input(f"Nós de conexão da barra {k} (nó_inicial
nó_final): ").split()
    barra.append((int(conn_str[0]), int(conn_str[1])))

Fx, Fy = [], []
for i in range(N):
    forces_str = input(f"Forças no Nó {i} (Fx Fy): ").split()
    Fx.append(float(forces_str[0]))
    Fy.append(float(forces_str[1]))

apoios = []
print("Tipos de apoio: 0=livre, 1=móvel (restrição em y), 2=fixo
(restrição em x e y)")
for i in range(N):
    apoios.append(int(input(f"Apoio no nó {i}: ")))

# =====
# Cálculos
# =====

A, b = montar_matriz_equilibrio(N, NB, x, y, barra, Fx, Fy, apoios)
sol = gauss_jordan([row[:] for row in A], b[:])

forcas_barras = sol[:NB]
classificacao = classificar_barras(forcas_barras)

print("\n--- Resultados ---")
print("Forças nas barras:", forcas_barras)
print("Classificação:", classificacao)
barra_max = np.argmax(np.abs(forcas_barras))
print("Barra mais solicitada:\n", barra_max)
print("Força média:\n", np.mean(forcas_barras))

```

```

print("Somatório das reações:\n", sum(sol[NB:]))

# =====
# Gráficos
# =====

# Treliça original
plt.figure(figsize=(10,4))
for k,(i,j) in enumerate(barra):
    plt.plot([x[i],x[j]], [y[i],y[j]], 'k-')
for i in range(N):
    plt.text(x[i],y[i]+0.05,f"N{i}")
plt.title("Treliça Original")
plt.axis('equal')
plt.show()

# Treliça colorida por intensidade
plt.figure(figsize=(10,4))
for k,(i,j) in enumerate(barra):
    cor = 'blue' if forcas_barras[k]>0 else 'red'

plt.plot([x[i],x[j]], [y[i],y[j]], color=cor, linewidth=2+abs(forcas_barras[k]))
plt.title("Treliça Colorida por Força")
plt.axis('equal')
plt.show()

# Gráfico de barras
plt.figure()
plt.bar(range(NB), np.abs(forcas_barras))
plt.title("Magnitude das Forças em Cada Barra")
plt.xlabel("Barra")
plt.ylabel("Força")
plt.show()

# Histograma
plt.figure()
plt.hist(forcas_barras, bins=10)
plt.title("Distribuição das Forças Internas")
plt.xlabel("Força")
plt.ylabel("Frequência")
plt.show()

```

Número de nós: 4
Coordenadas do Nó 0 (x y): 0 0
Coordenadas do Nó 1 (x y): 4 0
Coordenadas do Nó 2 (x y): 2 2
Coordenadas do Nó 3 (x y): 2 0
Número de barras: 5
Nós de conexão da barra 0 (nó_inicial nó_final): 0 2
Nós de conexão da barra 1 (nó_inicial nó_final): 2 1
Nós de conexão da barra 2 (nó_inicial nó_final): 0 3
Nós de conexão da barra 3 (nó_inicial nó_final): 3 1
Nós de conexão da barra 4 (nó_inicial nó_final): 2 3
Forças no Nó 0 (Fx Fy): 0 0
Forças no Nó 1 (Fx Fy): 0 0
Forças no Nó 2 (Fx Fy): 0 0
Forças no Nó 3 (Fx Fy): 0 -1000
Tipos de apoio: 0=livre, 1=móvel (restrição em y), 2=fixo (restrição em x e y)
Apoio no nó 0: 2
Apoio no nó 1: 1
Apoio no nó 2: 0
Apoio no nó 3: 0

--- Resultados ---
Forças nas barras: [707.1067811865476, 707.1067811865476, -500.0, -500.0, -1000.0]
Classificação: ['+', '+', '-', '-', '-']
Barra mais solicitada:
4
Força média:
-117.15728752538098
Somatório das reações:
-1000.0



